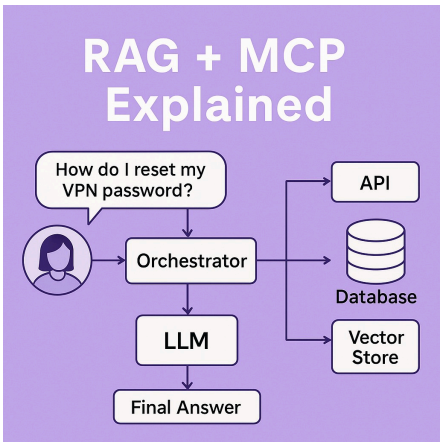
RAG - Retrieval-Augmented Generation

- RAG = **Retrieval + Generation**
- **LLM does not** rely only on **pre-trained memory**
- It **retrieves** relevant **data** in **real time** and then **generates** an **answer** using that data



Full-pipeline

1. Data Ingestion

- Collect **data** (PDFs, internal docs, databases)
- Convert to **text**
- Split into **chunks**

```
Chunk 1: Employees get 20 paid leaves per year
Chunk 2: Sick leave policy allows 10 days
```

2. Embedding Creation

- Convert each **chunk** into a **vector** (embedding)

3. Store in Vector Database

- Store both **text** and **embedding**

```
"Employees get 20 paid leaves" → [0.21, 0.78, ...]
"Sick leave policy..." → [0.65, 0.12, ...]
```

4. User Query

- User asks:

"How many leaves do I get?"

```
{
  text: "Employees get 20 paid leaves per year",
  embedding: [0.21, 0.78, ...]
}
```

5. Query Embedding

- Convert **query** into **vector**

```
"How many leaves do I get?" → [0.23, 0.81, ...]
```

6. Retrieval (Similarity Search)

- Compare **query embedding** with **stored embeddings**
- **Retrieve** most **relevant chunks**

```
"Employees get 20 paid leaves per year"
```

7. Prompt Augmentation

- Build final **prompt**.

8. Generation

- **LLM** generates answer:

"You get 20 paid leaves per year."

```
Context:
Employees get 20 paid leaves per year

Question:
How many leaves do I get?
```

